iHelp — Detailed Technical Design

This document describes the implemented system. The SQL here *is* the schema, and every RLS policy carries the product-spec rule it enforces.

---

1. Database Schema

1.1 Enums

```
create type public.request_status  as enum
  ('open','has_offers','assigned','completed','rated','cancelled');
create type public.offer_status    as enum
  ('active','selected','closed','withdrawn');
create type public.pricing_mode     as enum ('fixed','volunteer','after_job');
create type public.application_kind   as enum ('identity','professional');
create type public.application_status as enum
  ('pending','approved','rejected','revoked');
```

Enums over `text + CHECK`: the state machine values are closed sets that the whole design leans on; an enum makes an invalid state a *type error*, and the enum definition is self-documenting in any DB inspector. (Trade-off accepted: adding a value needs a migration — fine, the sets are stable by design.)

One deliberate exception: `help_requests.category` stays `text + CHECK`. It is *content*, not a state machine — a list expected to grow with the product — and `text + CHECK` keeps additions a small data-shaped migration. The app-side canonical list is `lib/categories.ts` (keys + Hebrew labels); the DB CHECK mirrors its keys.

1.2 Tables

```
-- Public profile: everything here is readable by any signed-in user.
create table public.profiles (
  id                  uuid primary key references auth.users(id) on delete cascade,
  display_name        text not null default ''
                      check (char_length(display_name) <= 40),
  is_identity_verified boolean not null default false,   -- set only by review_application /
revoke_verification
  is_professional     boolean not null default false,   -- set only by review_application /
revoke_verification
  created_at          timestamptz not null default now()
);

-- Private profile: own-row access only. Separate table because Postgres RLS is
-- row-level; these columns must never ride along with the broadly-readable row.
create table public.profiles_private (
  user_id    uuid primary key references public.profiles(id) on delete cascade,
  phone      text check (phone is null or phone ~ '^0\d{8,9}$'),
  lat        double precision check (lat between -90 and 90),
  lng        double precision check (lng between -180 and 180),
  constraint location_all_or_none check ((lat is null) = (lng is null)),
  is_admin   boolean not null default false              -- set only manually in SQL
);

create table public.verification_applications (
  id               uuid primary key default gen_random_uuid(),
  user_id          uuid not null references public.profiles(id) on delete cascade,
  kind             public.application_kind not null,
  status           public.application_status not null default 'pending',
```

```
  full_name        text not null check (char_length(full_name) between 2 and 60),
  self_description text not null default '' check (char_length(self_description) <= 500),
  -- the phone is part of the reviewed identity application (spec §8.2) — the
  -- admin must be able to see it via applications_select; on approval it is
  -- copied to profiles_private by review_application
  phone            text check (phone is null or phone ~ '^0\d{8,9}$'),
  constraint identity_requires_phone check (kind <> 'identity' or phone is not null),
  doc_path         text,            -- ID photo / certificate in verification-docs
  constraint professional_requires_doc check (kind <> 'professional' or doc_path is not null),
  admin_note       text,
  decided_by       uuid references public.profiles(id),
  decided_at       timestamptz,
  created_at       timestamptz not null default now()
);

-- Spec §9.2: at most one pending-or-approved application per user per kind.
-- Rejected/revoked rows stay behind as the audit trail and do not block re-apply.
create unique index one_open_application_per_kind
  on public.verification_applications (user_id, kind)
  where status in ('pending','approved');

create table public.help_requests (
  id            uuid primary key default gen_random_uuid(),
  requester_id  uuid not null references public.profiles(id) on delete cascade,
  title         text not null check (char_length(title) between 3 and 80),
  description   text not null check (char_length(description) between 10 and 2000),
  category      text not null check (category in
                  ('repairs','electricity','plumbing','moving','tutoring',
                   'tech_help','errands','gardening','pets','other')),
  -- request location, confirmed by the requester at publish time — NOT NULL:
  -- the spec (C3, §8.3, §9.3) makes location part of every request; a request
  -- helpers cannot locate defeats the distance-sorted marketplace (G3). The
  -- posting form captures/confirms it (profile default or on-the-spot prompt).
  lat           double precision not null check (lat between -90 and 90),
  lng           double precision not null check (lng between -180 and 180),
  status        public.request_status not null default 'open',
  is_hidden     boolean not null default false,          -- admin moderation flag
  assigned_offer_id      uuid,                           -- FK added below (circular)
  completed_by_requester boolean not null default false,
  completed_by_helper    boolean not null default false,
  is_paid       boolean not null default false,          -- owner's marker, via mark_paid RPC
  created_at    timestamptz not null default now(),
  assigned_at   timestamptz,
  completed_at  timestamptz,
  rated_at      timestamptz,
  cancelled_at  timestamptz
);

create table public.offers (
  id             uuid primary key default gen_random_uuid(),
  request_id     uuid not null references public.help_requests(id) on delete cascade,
  helper_id      uuid not null references public.profiles(id) on delete cascade,
  status         public.offer_status not null default 'active',
  message        text not null check (char_length(message) between 5 and 1000),
  -- three pricing stances: a helper often cannot quote before
  -- seeing the problem. pricing_mode:
  --   'fixed'     → price set now (price column)
  --   'volunteer' → free (both price columns null)
  --   'after_job' → priced once the work is done (final_price set later via the
  --                 set_final_price RPC, when the request is completed)
  pricing_mode   public.pricing_mode not null default 'volunteer',
  price          numeric(10,2) check (price is null or (price > 0 and price <= 99999.99)),
  final_price    numeric(10,2) check (final_price is null or (final_price > 0 and final_price <=
99999.99)),
  constraint price_matches_mode check (
    (pricing_mode = 'fixed'     and price is not null) or
    (pricing_mode = 'volunteer' and price is null and final_price is null) or
    (pricing_mode = 'after_job' and price is null)
  ),
```

```
  -- any of the three stances is allowed on any request.
  -- snapshot set by trigger T4 at insert: /my/offers must render meaningfully
  -- even after the offerer loses SELECT on the parent request (spec §9.2)
  request_title  text not null default '',
  created_at     timestamptz not null default now()
);

-- Spec §9.2: one *active* offer per helper per request. Withdrawn/closed rows
-- do not block a new offer (withdraw-then-reoffer is allowed while open).
create unique index one_active_offer_per_helper
  on public.offers (request_id, helper_id) where (status = 'active');

-- Circular FK, added after both tables exist:
alter table public.help_requests
  add constraint fk_assigned_offer
  foreign key (assigned_offer_id) references public.offers(id);

create table public.request_photos (
  id           uuid primary key default gen_random_uuid(),
  request_id   uuid not null references public.help_requests(id) on delete cascade,
  storage_path text not null,
  position     int not null default 0,
  created_at   timestamptz not null default now()
);

create table public.ratings (
  -- PK on request_id: one rating per request, by construction (spec §9.2)
  request_id uuid primary key references public.help_requests(id) on delete cascade,
  helper_id  uuid not null references public.profiles(id) on delete cascade,
  rater_id   uuid not null references public.profiles(id) on delete cascade,
  stars      int not null check (stars between 1 and 5),
  note       text check (note is null or char_length(note) <= 500),
  created_at timestamptz not null default now()
);
```

Rating aggregates are computed, not stored. Helper profile pages and offer lists read `avg(stars)`, `count(*)` grouped by `helper_id` in one query. At MVP scale this is trivially cheap, always correct, and has no update anomaly; denormalized counters are the named successor in the scale document.

1.3 Indexes (beyond PKs and the two partial uniques)

```
create index idx_requests_browse  on public.help_requests (status, is_hidden, created_at desc);
create index idx_requests_owner   on public.help_requests (requester_id, created_at desc);
create index idx_offers_request   on public.offers (request_id) where status = 'active';
create index idx_offers_helper    on public.offers (helper_id, created_at desc);
create index idx_photos_request   on public.request_photos (request_id, position);
create index idx_ratings_helper   on public.ratings (helper_id);
create index idx_applications_queue on public.verification_applications (status, created_at)
  where status = 'pending';
```

Each index maps to a page: browse feed, my-requests, request detail (offers + photos), my-offers, helper profile, admin queue.

---

2. Row Level Security — every policy, with its justification

RLS is enabled on all seven tables (`alter table … enable row level security`). No table grants anything to `anon` — every policy targets `authenticated`. There is deliberately **no INSERT policy** on `help_requests`, `request_photos`, and `ratings`: those inserts happen only inside SECURITY DEFINER RPCs, which is what makes their invariants (atomic status flip) unskippable.

Helper function

```
-- SECURITY DEFINER so policies can check adminship without profiles_private
-- being readable; STABLE so the planner caches it per statement.
create or replace function public.is_admin() returns boolean
language sql stable security definer set search_path = public as $$
  select coalesce(
    (select is_admin from public.profiles_private where user_id = auth.uid()),
    false);
$$;

create or replace function public.is_identity_verified() returns boolean
language sql stable security definer set search_path = public as $$
  select coalesce(
    (select is_identity_verified from public.profiles where id = auth.uid()),
    false);
$$;

-- Breaks the policy recursion cycle: help_requests' SELECT policy must ask
-- "is the caller the selected helper?", which reads offers — but offers'
-- SELECT policy reads help_requests back. Cross-referencing policies recurse
-- ("infinite recursion detected in policy"); a SECURITY DEFINER lookup on one
-- side terminates the chain.
create or replace function public.is_selected_helper(p_assigned_offer_id uuid)
returns boolean
language sql stable security definer set search_path = public as $$
  select exists (
    select 1 from public.offers o
    where o.id = p_assigned_offer_id and o.helper_id = auth.uid());
$$;
```

`profiles`

Policy	SQL condition	Enforces (spec)
`profiles_select` (SELECT)	`true` (authenticated)	Helper cards, offer lists, and `/helpers/[id]` show any user's name + badges — public-by-design columns only (§4.3)
`profiles_update_own` (UPDATE)	USING/CHECK `id = auth.uid()`	Only you edit your profile; the column-guard trigger (§4 below) keeps the two verification flags out of reach

No INSERT/DELETE policies — rows are created by the signup trigger and die with the `auth.users` cascade.

`profiles_private`

Policy	SQL condition	Enforces
`private_select_own` (SELECT)	`user_id = auth.uid()`	Phone and home coordinates are readable by their owner only (§9.3); the contact RPC is the sole cross-user path
`private_update_own` (UPDATE)	USING/CHECK `user_id = auth.uid()`	Owner edits phone/location; `is_admin` is column-guard-protected — **without the guard, this policy would let any user set `is_admin = true` on their own row**. The guard also blocks *removing* a phone once set (change allowed, null not) — the contact-reveal flow must never surface an empty phone for a verified user

`verification_applications`

Policy	SQL condition	Enforces
`applications_select` (SELECT)	`user_id = auth.uid() or public.is_admin()`	Applicant sees own history + status; admins see the queue (§9.2)
`applications_insert` (INSERT)	`user_id = auth.uid() and (kind = 'identity' or public.is_identity_verified()) and status = 'pending' and admin_note is null and decided_by is null and decided_at is null and (doc_path is null or doc_path like auth.uid()::text || '/%')`	Anyone applies for identity; professional requires approved identity (§9.2); the partial unique index blocks a second open application. The `status` / `decided_*` pins make "decisions go through `review_application` only" true *by construction* — without them a crafted insert forges an already-approved audit row; the `doc_path` prefix pin stops referencing someone else's ID photo/certificate

No UPDATE/DELETE for users: applications are immutable once submitted (re-apply creates a new row — that *is* the audit trail); decisions go through `review_application` only.

`help_requests`

Policy	SQL condition	Enforces
`requests_select` (SELECT)	`(status in ('open','has_offers') and not is_hidden) or requester_id = auth.uid() or public.is_admin() or public.is_selected_helper(assigned_offer_id)`	The §9.2 view rule verbatim: everyone sees the live feed minus hidden; owner, selected helper, and admins also see later states and hidden rows. The selected-helper check goes through the definer helper to avoid policy recursion with `offers`
`requests_update_own` (UPDATE)	USING `requester_id = auth.uid() and status in ('open','has_offers')` CHECK `requester_id = auth.uid()`	Owner edits content while editable (§9.2); which *columns* may change is the guard trigger's job. **This is deliberately the only UPDATE policy on the table**: permissive policies OR their USING and WITH CHECK clauses independently, so a second "mark paid" policy would let a completed-paid request pass its USING while the content-edit policy's lax CHECK accepts arbitrary new content — reopening edits on finished jobs. The paid marker therefore goes through the `mark_paid` RPC instead

No INSERT (RPC only), no DELETE (cancellation is a state, not a row removal — offers and ratings reference the row forever).

`offers`

Policy	SQL condition	Enforces
`offers_select` (SELECT)	`helper_id = auth.uid() or exists (select 1 from public.help_requests r where r.id = request_id and r.requester_id = auth.uid())`	Sealed-bid visibility: owner of the offer + owner of the request, nobody else — including admins (§9.2)
`offers_insert` (INSERT)	`helper_id = auth.uid() and status = 'active' and final_price is null and public.is_identity_verified() and exists (select 1 from public.help_requests r where r.id = request_id and r.requester_id <> auth.uid() and r.status in ('open','has_offers') and not r.is_hidden)`	Verified users only; not on own request; any pricing stance on any request; only while the request is open/has_offers and visible (§9.2); duplicate active blocked by the partial unique index. **`status = 'active'` pins birth state** — without it a crafted insert creates an offer born `selected` (spoofing the requester's comparison view and surviving `assign_offer`'s active-only sweep) or born `closed` / `withdrawn` (evading the uniqueness index). **`final_price is null` pins insert** — without it a crafted after_job insert fabricates the "agreed" amount `mark_paid` coalesces to, bypassing the entire `set_final_price` guard chain
`offers_update_own` (UPDATE)	USING `helper_id = auth.uid() and status = 'active'` CHECK `helper_id = auth.uid() and status in ('active','withdrawn')`	Edit or withdraw while active. The CHECK's closed set is what stops a helper PATCHing their own offer to `selected` — the only two states a helper can write are the two they own (§9.2). `request_id`, `helper_id`, `pricing_mode`, `final_price`, `created_at` are column-guard-protected (below) — otherwise an UPDATE could *re-point* an active offer at a different request, bypassing every INSERT-time check (own-request, open-status, hidden), or write `final_price` directly instead of via `set_final_price`

No DELETE — withdrawn offers stay as history (and as re-offer bookkeeping).

`request_photos`

Policy	SQL condition	Enforces
`photos_select` (SELECT)	`exists (select 1 from public.help_requests r where r.id = request_id)`	Mirrors the parent request's visibility automatically: the subquery is itself RLS-filtered per caller, so photos of hidden/closed requests disappear for exactly the users the request disappears for

No INSERT/UPDATE/DELETE: photos are created by the RPC and are **immutable afterwards** — a deliberate MVP simplification (the edit flow changes text fields only, spec-consistent).

`ratings`

Policy	SQL condition	Enforces
`ratings_select` (SELECT)	`helper_id = auth.uid() or rater_id = auth.uid() or public.is_admin()`	The *base table* is party-scoped: it carries `rater_id` + `request_id`, and a `true` policy would let any signed-in user dump who-rated-whom platform-wide — linkage the parent (rated, RLS-invisible) request keeps hidden. Third parties read ratings through the view below

```
-- The public rating surface (spec §9.2 "View rating | any signed-in user"):
-- stars + note + when, per helper — WITHOUT rater/request linkage. Postgres
-- views execute with the owner's rights by default, which is exactly the
-- column-slicing tool RLS lacks. Rater identity is deliberately not shown to
-- third parties; the helper can infer it from the request context anyway.
create view public.helper_ratings
  with (security_invoker = false) as
  select helper_id, stars, note, created_at from public.ratings;
grant select on public.helper_ratings to authenticated;
```

No INSERT (RPC only — the insert must atomically advance the request to *rated*), no UPDATE/DELETE (immutable, spec §9.1).

Storage policies

```
-- bucket: request-photos (private bucket; policies on storage.objects)
create policy "photos_upload" on storage.objects for insert to authenticated
  with check (bucket_id = 'request-photos'
    and (storage.foldername(name))[1] = auth.uid()::text
    and public.is_identity_verified());
create policy "photos_read" on storage.objects for select to authenticated
  using (bucket_id = 'request-photos');

-- bucket: verification-docs
create policy "docs_upload" on storage.objects for insert to authenticated
  with check (bucket_id = 'verification-docs'
    and (storage.foldername(name))[1] = auth.uid()::text);
create policy "docs_read" on storage.objects for select to authenticated
  using (bucket_id = 'verification-docs'
    and ((storage.foldername(name))[1] = auth.uid()::text or public.is_admin()));
```

Bucket-level config: 5 MB object size limit, MIME allowlist (`image/jpeg`, `image/png`, `image/webp`). The known read-scope gap on `request-photos` (objects of hidden requests remain fetchable by path) is the accepted limitation documented in architecture §5.

---

3. Database Functions — full bodies (the privileged-code inventory)

Conventions for all eleven: `security definer set search_path = public`, execute revoked from `public`/`anon` and granted to `authenticated`, permission checks first, business errors raised with stable codes the app maps to Hebrew (`P0001` + message in `not_found | forbidden | invalid_state | …`).

Error-ordering rule (no existence leaks): RPCs read with definer rights, so a naive check order would reveal rows RLS hides — e.g., raising `invalid_state` for a *hidden* request tells a probing caller the row exists. The rule: **raise `not_found` whenever the caller could not SELECT the row under the policies** (non-owner, non-party), *before* any state check; `invalid_state` and `forbidden` are only ever raised to callers who can already see the row. This keeps the §10 promise — denied and missing are indistinguishable.

```
-- 3.1 Create request + photos atomically; photos optional (0–5) and, when
-- supplied, path ownership is enforced.
create or replace function public.create_request_with_photos(
  p_title text, p_description text, p_category text,
  p_lat double precision, p_lng double precision,
  p_photo_paths text[]
) returns uuid
language plpgsql security definer set search_path = public as $$
declare
  v_id uuid;
  v_path text;
  v_paths text[];
begin
  if not public.is_identity_verified() then
    raise exception 'forbidden';
  end if;
  if p_lat is null or p_lng is null then
    raise exception 'location_required';
  end if;

  -- deduplicate, then bound: 0–5 distinct photos (photos optional)
  select array_agg(distinct p) into v_paths from unnest(p_photo_paths) as p;
  if array_length(v_paths, 1) > 5 then
    raise exception 'too_many_photos';
  end if;
  foreach v_path in array v_paths loop
    -- photos must live in the caller's own storage folder
    if v_path not like auth.uid()::text || '/%' then
      raise exception 'forbidden';
    end if;
  end loop;
  -- every path must be a real object in the right bucket (definer read of
  -- storage.objects): blocks nonexistent paths and verification-docs paths,
  -- which share the same {uid}/ folder convention
  if (select count(*) from storage.objects
      where bucket_id = 'request-photos' and name = any(v_paths))
     <> array_length(v_paths, 1) then
    raise exception 'photo_not_uploaded';
  end if;

  insert into public.help_requests
    (requester_id, title, description, category, lat, lng)
  values
    (auth.uid(), p_title, p_description, p_category, p_lat, p_lng)
  returning id into v_id;

  insert into public.request_photos (request_id, storage_path, position)
  select v_id, u.path, u.ord - 1
  from unnest(v_paths) with ordinality as u(path, ord);

  return v_id;
end $$;

-- 3.2 Assign: the pivotal moment. Guarded updates close the withdraw race;
-- the FOR UPDATE lock serializes concurrent assigns.
create or replace function public.assign_offer(p_request_id uuid, p_offer_id uuid)
returns void
language plpgsql security definer set search_path = public as $$
declare v_requester uuid;
begin
```

```
  select requester_id into v_requester
    from public.help_requests where id = p_request_id for update;
  -- error-ordering rule: non-owner gets not_found, same as a missing row
  if v_requester is null or v_requester <> auth.uid() then
    raise exception 'not_found';
  end if;

  update public.help_requests
     set status = 'assigned', assigned_offer_id = p_offer_id, assigned_at = now()
   where id = p_request_id and status = 'has_offers';
  if not found then raise exception 'invalid_state'; end if;

  update public.offers
     set status = 'selected'
   where id = p_offer_id and request_id = p_request_id and status = 'active';
  if not found then raise exception 'offer_not_active'; end if;  -- rolls back both

  update public.offers
     set status = 'closed'
   where request_id = p_request_id and status = 'active';
end $$;

-- 3.3 Dual-sided completion: caller's side derived from identity, never a parameter.
create or replace function public.confirm_completion(p_request_id uuid)
returns void
language plpgsql security definer set search_path = public as $$
declare
  v_req public.help_requests%rowtype;
  v_helper uuid;
begin
  select * into v_req from public.help_requests
    where id = p_request_id for update;
  if not found then raise exception 'not_found'; end if;

  select helper_id into v_helper from public.offers where id = v_req.assigned_offer_id;
  -- error-ordering rule: party check BEFORE state check — a non-party probing
  -- a hidden/cancelled id must learn nothing, not even "wrong state"
  if auth.uid() <> v_req.requester_id and (v_helper is null or auth.uid() <> v_helper) then
    raise exception 'not_found';
  end if;
  if v_req.status <> 'assigned' then raise exception 'invalid_state'; end if;

  if auth.uid() = v_req.requester_id then
    update public.help_requests set completed_by_requester = true where id = p_request_id;
  else
    update public.help_requests set completed_by_helper = true where id = p_request_id;
  end if;

  update public.help_requests
     set status = 'completed', completed_at = now()
   where id = p_request_id and status = 'assigned'
     and completed_by_requester and completed_by_helper;
end $$;

-- 3.4 Cancel: owner-only, terminal, closes all live offers (cross-owner writes).
create or replace function public.cancel_request(p_request_id uuid)
returns void
language plpgsql security definer set search_path = public as $$
declare v_requester uuid;
begin
  select requester_id into v_requester
    from public.help_requests where id = p_request_id for update;
  if v_requester is null or v_requester <> auth.uid() then
    raise exception 'not_found';   -- error-ordering rule
  end if;

  update public.help_requests
     set status = 'cancelled', cancelled_at = now()
   where id = p_request_id and status in ('open','has_offers','assigned');
```

```
  if not found then raise exception 'invalid_state'; end if;

  update public.offers
     set status = 'closed'
   where request_id = p_request_id and status in ('active','selected');
end $$;

-- 3.5 Rating: insert + completed→rated flip in one transaction.
create or replace function public.submit_rating(
  p_request_id uuid, p_stars int, p_note text
) returns void
language plpgsql security definer set search_path = public as $$
declare
  v_req public.help_requests%rowtype;
  v_helper uuid;
begin
  select * into v_req from public.help_requests
    where id = p_request_id for update;
  if not found or v_req.requester_id <> auth.uid() then
    raise exception 'not_found';   -- error-ordering rule
  end if;
  if v_req.status <> 'completed' then raise exception 'invalid_state'; end if;

  select helper_id into v_helper from public.offers where id = v_req.assigned_offer_id;

  insert into public.ratings (request_id, helper_id, rater_id, stars, note)
  values (p_request_id, v_helper, auth.uid(), p_stars, nullif(trim(p_note), ''));

  update public.help_requests
     set status = 'rated', rated_at = now()
   where id = p_request_id;
end $$;

-- 3.6 Admin: decide an application; flags update atomically with the decision.
create or replace function public.review_application(
  p_application_id uuid, p_approve boolean, p_note text
) returns void
language plpgsql security definer set search_path = public as $$
declare v_app public.verification_applications%rowtype;
begin
  if not public.is_admin() then raise exception 'forbidden'; end if;

  select * into v_app from public.verification_applications
    where id = p_application_id for update;
  if not found then raise exception 'not_found'; end if;
  if v_app.status <> 'pending' then raise exception 'invalid_state'; end if;
  -- a rejection must carry a reason (spec §4.1: "rejects with a note")
  if not p_approve and nullif(trim(p_note), '') is null then
    raise exception 'note_required';
  end if;
  -- a professional badge on a revoked identity is meaningless: re-check the
  -- gate at decision time, not just at application time
  if p_approve and v_app.kind = 'professional' and not exists (
    select 1 from public.profiles
    where id = v_app.user_id and is_identity_verified
  ) then
    raise exception 'invalid_state';
  end if;

  update public.verification_applications
     set status     = case when p_approve then 'approved' else 'rejected' end,
         admin_note = p_note,
         decided_by = auth.uid(),
         decided_at = now()
   where id = p_application_id;

  if p_approve then
    if v_app.kind = 'identity' then
      update public.profiles set is_identity_verified = true where id = v_app.user_id;
```

```
      -- the reviewed phone becomes the live contact channel (spec §8.2)
      update public.profiles_private set phone = v_app.phone where user_id = v_app.user_id;
    else
      update public.profiles set is_professional = true where id = v_app.user_id;
    end if;
  end if;
end $$;

-- 3.7 Admin: revoke. Identity revocation also drops the professional badge
-- (professional requires identity, spec §9.2).
create or replace function public.revoke_verification(
  p_user_id uuid, p_kind public.application_kind
) returns void
language plpgsql security definer set search_path = public as $$
begin
  if not public.is_admin() then raise exception 'forbidden'; end if;

  update public.verification_applications
     set status = 'revoked', decided_by = auth.uid(), decided_at = now()
   where user_id = p_user_id and kind = p_kind and status = 'approved';
  if not found then raise exception 'not_found'; end if;

  if p_kind = 'identity' then
    update public.profiles
       set is_identity_verified = false, is_professional = false
     where id = p_user_id;
    -- professional rides on identity: revoke approved AND pending professional
    -- applications — otherwise a surviving pending row could later be approved
    -- onto a revoked identity
    update public.verification_applications
       set status = 'revoked', decided_by = auth.uid(), decided_at = now()
     where user_id = p_user_id and kind = 'professional'
       and status in ('approved','pending');
  else
    update public.profiles set is_professional = false where id = p_user_id;
  end if;
end $$;

-- 3.8 Admin: moderation flag only — lifecycle state untouched by construction.
create or replace function public.set_request_hidden(
  p_request_id uuid, p_hidden boolean
) returns void
language plpgsql security definer set search_path = public as $$
begin
  if not public.is_admin() then raise exception 'forbidden'; end if;
  update public.help_requests set is_hidden = p_hidden where id = p_request_id;
  if not found then raise exception 'not_found'; end if;
end $$;

-- 3.9 Paid marker: RPC rather than an UPDATE policy on purpose — a second
-- permissive UPDATE policy on help_requests would OR its USING with the
-- content-edit policy's lax CHECK and reopen content edits on finished jobs.
create or replace function public.mark_paid(p_request_id uuid)
returns void
language plpgsql security definer set search_path = public as $$
declare
  v_req public.help_requests%rowtype;
  v_offer public.offers%rowtype;
  v_agreed numeric;
begin
  select * into v_req from public.help_requests where id = p_request_id for update;
  if not found or v_req.requester_id <> auth.uid() then
    raise exception 'not_found';   -- error-ordering rule
  end if;
  -- keyed to the AGREED amount: a fixed quote OR a set after_job final price
  -- (a volunteered job has nothing to mark as paid)
  select * into v_offer from public.offers where id = v_req.assigned_offer_id;
  v_agreed := coalesce(v_offer.price, v_offer.final_price);  -- fixed OR set after_job
  if v_req.status not in ('completed','rated')
```

```
     or v_agreed is null or v_req.is_paid then
    raise exception 'invalid_state';
  end if;
  update public.help_requests set is_paid = true where id = p_request_id;
end $$;

-- 3.10 After-job pricing: the selected helper of an
-- `after_job` offer sets the final amount once the request is completed
-- (or rated — the requester may still mark paid). Guarded: selected helper
-- only, after_job only, once.
create or replace function public.set_final_price(p_request_id uuid, p_price numeric)
returns void
language plpgsql security definer set search_path = public as $$
declare
  v_req public.help_requests%rowtype;
  v_offer public.offers%rowtype;
begin
  if p_price is null or p_price <= 0 or p_price > 99999.99 then
    raise exception 'invalid_price';
  end if;

  select * into v_req from public.help_requests where id = p_request_id for update;
  if not found then raise exception 'not_found'; end if;

  select * into v_offer from public.offers where id = v_req.assigned_offer_id;
  -- error-ordering rule: only the selected helper can price; anyone else 404s
  if v_offer.helper_id is null or v_offer.helper_id <> auth.uid() then
    raise exception 'not_found';
  end if;
  if v_req.status not in ('completed','rated')
     or v_offer.pricing_mode <> 'after_job'
     or v_offer.final_price is not null then
    raise exception 'invalid_state';
  end if;

  update public.offers set final_price = p_price where id = v_offer.id;
end $$;

-- 3.11 The only read RPC: counterpart contact, post-assignment, parties only.
create or replace function public.get_counterpart_contact(p_request_id uuid)
returns table (display_name text, phone text)
language plpgsql security definer set search_path = public as $$
declare
  v_req public.help_requests%rowtype;
  v_helper uuid;
  v_other uuid;
begin
  select * into v_req from public.help_requests where id = p_request_id;
  if not found then raise exception 'not_found'; end if;

  select helper_id into v_helper from public.offers where id = v_req.assigned_offer_id;
  -- error-ordering rule: party check first — probing a hidden/cancelled id
  -- must not reveal that the row exists or what state it is in
  if auth.uid() = v_req.requester_id then v_other := v_helper;
  elsif v_helper is not null and auth.uid() = v_helper then v_other := v_req.requester_id;
  else raise exception 'not_found';
  end if;

  if v_req.status not in ('assigned','completed','rated') then
    raise exception 'invalid_state';
  end if;

  return query
    select p.display_name, pp.phone
    from public.profiles p
    join public.profiles_private pp on pp.user_id = p.id
    where p.id = v_other;
end $$;
```

Triggers (four functions)

```
-- T1: signup — create both profile rows.
create or replace function public.handle_new_user() returns trigger
language plpgsql security definer set search_path = public as $$
begin
  insert into public.profiles (id) values (new.id);
  insert into public.profiles_private (user_id) values (new.id);
  return new;
end $$;
create trigger on_auth_user_created after insert on auth.users
  for each row execute function public.handle_new_user();

-- T2: offer lifecycle keeps request open ↔ has_offers true, and closes the
-- race where an offer lands on a just-assigned request.
create or replace function public.sync_request_offer_status() returns trigger
language plpgsql security definer set search_path = public as $$
declare
  v_request_id uuid := coalesce(new.request_id, old.request_id);
  v_status public.request_status;
  v_active int;
begin
  select status into v_status from public.help_requests
    where id = v_request_id for update;              -- waits out concurrent assign/cancel

  if v_status in ('open','has_offers') then
    select count(*) into v_active from public.offers
      where request_id = v_request_id and status = 'active';
    update public.help_requests
       set status = case when v_active > 0 then 'has_offers' else 'open' end
     where id = v_request_id and status <> (case when v_active > 0
       then 'has_offers'::public.request_status else 'open'::public.request_status end);
  elsif tg_op = 'INSERT' and new.status = 'active' then
    -- request left the offerable states while this insert was in flight
    update public.offers set status = 'closed' where id = new.id;
  end if;
  return new;
end $$;
create trigger on_offer_change after insert or update of status on public.offers
  for each row execute function public.sync_request_offer_status();

-- T3: column guard — RLS cannot restrict WHICH columns an UPDATE changes.
-- Direct (PostgREST) writes run as role 'authenticated'; the definer RPCs run
-- as the function owner. The guard rejects protected-column changes for
-- 'authenticated' and lets the privileged path through.
-- NOTE: deliberately *invoker-rights* (not SECURITY DEFINER) — the mechanism
-- depends on current_user being the CALLER's role; search_path pinned anyway.
create or replace function public.guard_protected_columns() returns trigger
language plpgsql set search_path = public as $$
begin
  if current_user <> 'authenticated' then
    return new;                                       -- privileged path (RPCs, SQL console)
  end if;

  if tg_table_name = 'profiles' then
    if new.is_identity_verified is distinct from old.is_identity_verified
       or new.is_professional  is distinct from old.is_professional then
      raise exception 'forbidden';
    end if;
  elsif tg_table_name = 'profiles_private' then
    if new.is_admin is distinct from old.is_admin then
      raise exception 'forbidden';
    end if;
    -- a phone may be changed but never removed once set: the contact-reveal
    -- flow must not surface an empty phone for a verified user
    if old.phone is not null and new.phone is null then
      raise exception 'forbidden';
    end if;
```

```
  elsif tg_table_name = 'offers' then
    if new.request_id is distinct from old.request_id
       or new.helper_id is distinct from old.helper_id
       or new.request_title is distinct from old.request_title
       or new.pricing_mode is distinct from old.pricing_mode
       or new.final_price is distinct from old.final_price   -- set_final_price RPC only
       or new.created_at is distinct from old.created_at then
      raise exception 'forbidden';
    end if;
  elsif tg_table_name = 'help_requests' then
    if new.status is distinct from old.status
       or new.is_hidden is distinct from old.is_hidden
       or new.assigned_offer_id is distinct from old.assigned_offer_id
       or new.completed_by_requester is distinct from old.completed_by_requester
       or new.completed_by_helper is distinct from old.completed_by_helper
       or new.requester_id is distinct from old.requester_id
       or new.created_at is distinct from old.created_at
       or new.assigned_at is distinct from old.assigned_at
       or new.completed_at is distinct from old.completed_at
       or new.rated_at is distinct from old.rated_at
       or new.cancelled_at is distinct from old.cancelled_at
       or new.is_paid is distinct from old.is_paid then   -- is_paid: mark_paid RPC only
      raise exception 'forbidden';
    end if;
  end if;
  return new;
end $$;

create trigger guard_profiles before update on public.profiles
  for each row execute function public.guard_protected_columns();
create trigger guard_profiles_private before update on public.profiles_private
  for each row execute function public.guard_protected_columns();
create trigger guard_help_requests before update on public.help_requests
  for each row execute function public.guard_protected_columns();
create trigger guard_offers before update on public.offers
  for each row execute function public.guard_protected_columns();

-- T4: offer-insert preparation (invoker-rights: the parent request is visible
-- to the inserter by policy). Normalizes server-controlled fields and takes
-- the title snapshot /my/offers renders after the parent becomes invisible.
create or replace function public.prepare_offer_insert() returns trigger
language plpgsql set search_path = public as $$
begin
  new.created_at := now();                       -- never caller-supplied
  select title into new.request_title
    from public.help_requests where id = new.request_id;
  return new;
end $$;
create trigger on_offer_insert before insert on public.offers
  for each row execute function public.prepare_offer_insert();
```

4. CRUD Map (assignment: central CREATE/READ/UPDATE/DELETE)

Entity	Create	Read	Update	Delete
Profile (public+private)	Signup trigger	Public row: anyone signed-in; private row: owner	Owner (name, phone, location); flags via `review_application` / `revoke_verification` only	Cascade with account
Verification application	Applicant (INSERT policy)	Applicant + admins	Decision via `review_application` RPC only	Never (audit trail)
Help request	`create_request_with_photos` RPC	Feed rule / owner / selected helper / admin	Owner content-edit (open/has_offers); transitions via RPCs; `is_paid` via `mark_paid` RPC	Never — `cancelled` is a state; rows keep offer/rating history
Request photo	Same RPC (0–5)	Mirrors parent request	Never (immutable set)	Cascade with request
Offer	Helper (INSERT policy)	Offer owner + request owner	Owner edit/withdraw while active; `selected` / `closed` via RPCs	Never — withdrawn is a state
Rating	`submit_rating` RPC	Parties + admins on the base table; everyone else via the `helper_ratings` view (no rater linkage)	Never	Never

"Never" cells are decisions, not omissions: audit trails and referential history outweigh hard deletes everywhere in this domain (account deletion cascades are the one exception, delegated to Supabase's `auth.users` cascade).

---

5. API Description

The full action inventory is architecture §7. Contract details:

- **Transport:** Server Actions (POST, same-origin, Next.js encrypted action IDs). No public REST surface; the DB's PostgREST endpoint *is* reachable with the anon key but exposes only what RLS grants — that is the audited surface.
- **Action result type** (uniform):

```
type ActionResult<T = void> =
  | { ok: true; data: T }
  | { ok: false; formError?: string; fieldErrors?: Record<string, string> };
```

- **RPC error mapping** (Postgres `RAISE EXCEPTION` message → Hebrew UI string):

Code	Meaning	UI (Hebrew)
`not_found`	Row absent or invisible to caller	"הבקשה לא נמצאה"
`forbidden`	Caller lacks the right	"אין לך הרשאה לפעולה זו"
`invalid_state`	State machine forbids the transition	"הפעולה אינה זמינה במצב הנוכחי"
`offer_not_active`	Assign raced a withdrawal	"ההצעה כבר אינה זמינה — רעננו את העמוד"
`too_many_photos`	More than 5 photos supplied (photos are optional, 0–5)	"ניתן לצרף עד 5 תמונות"
`photo_not_uploaded`	A photo path has no uploaded object behind it	"העלאת התמונות נכשלה — נסו שוב"
`location_required`	Request published without coordinates	"יש לאשר מיקום לבקשה"
`invalid_price`	Final price out of range (`set_final_price`)	"סכום לא תקין"
`note_required`	Admin rejected without a reason	"דחייה מחייבת נימוק"
(RLS denial / no rows)	Permission denied at policy level	Same generic "אין הרשאה" — deliberately indistinguishable from not-found

Silent-denial pattern for direct updates: rows an UPDATE policy's USING clause filters out are *silently skipped* — PostgREST reports success with zero affected rows, not an error. Every direct-update action (`updateRequest`, `updateOffer`, `withdrawOffer`, `updateProfile`) therefore chains `.select()` and treats an empty result as a denial, mapped to the same generic message. Only WITH CHECK violations, constraint/trigger raises, and RPC raises arrive as Postgres errors.

- **Reads:** Server Components use the per-request Supabase client. The three read shapes worth naming: the feed (`status in ('open','has_offers') and not is_hidden` — matching `idx_requests_browse`; "open" in prose always means "not yet assigned" — capped 200, Haversine-sorted in `lib/geo.ts`, paginated in-memory — architecture §8.1), the request detail (request + photos + offers visible to caller + rating + contact RPC when assigned), and helper profile (public profile + aggregate and list from the `helper_ratings` view).
- **Image delivery:** both buckets are private, so `<img src>` cannot reference them directly. Server Components create **bulk signed URLs** (`storage.createSignedUrls`, one call per rendered page, 1-hour expiry — matching navigation-time freshness) for request photos and, on the admin queue, for verification documents. Signing runs as the signed-in user, so the same storage policies authorize it; URLs expire instead of living forever in the HTML.

---

6. Central Business Logic

One table — transition × actor × mechanism (the spec §9.1 machine, made operational):

Transition	Actor	Mechanism
→ `open` (publish)	Verified owner	`create_request_with_photos`
`open ↔ has_offers`	System	Trigger T2 on offer insert/withdraw
`has_offers → assigned`	Owner	`assign_offer` (locks, guards, closes competitors)
`assigned`: set own completion flag	Owner / selected helper	`confirm_completion` (side from `auth.uid()`)
`assigned → completed`	System (both flags true)	Same RPC, same transaction
`completed → rated`	Owner	`submit_rating` (insert + flip, atomic)
`after_job` final price	Selected helper	`set_final_price` (completed/rated, after_job offer, once)
`is_paid` marker	Owner	`mark_paid` (post-completion, agreed amount = `coalesce(price, final_price)`, once)
any pre-completed → `cancelled`	Owner	`cancel_request` (closes live offers)
`is_hidden` flip	Admin	`set_request_hidden`
Verification decide / revoke	Admin	`review_application` / `revoke_verification`

Distance: `lib/geo.ts` implements Haversine (`~15` lines, pure, unit-tested). Every request carries coordinates (NOT NULL — §1.2); the *viewer* may lack a location, in which case the feed renders newest-first without distances (spec C4 fallback). The viewer's own location comes from their `profiles_private` row (own-row read).

---

7. Components (central React components and their type)

Component	Kind	Notes
`(app)/layout`	Server	Nav, session read, onboarding redirect (empty display name → `/profile`)
`RequestCard`, `RequestList`	Server	Feed + my-requests rendering; distance chip
`RequestDetail`	Server	Composes photos, status panel, offers/rating/contact per role
`RequestForm`	Client	Create/edit; zod client-parse; wraps `PhotoUploader`
`PhotoUploader`	Client	Direct-to-storage upload, up to 5 files (optional), size/type checks, returns paths
`OfferList`, `OfferCard`	Server	Owner sees all offers + helper badges/ratings; helper sees own
`OfferForm`, `WithdrawButton`	Client	Offer create/edit/withdraw
`AssignButton`	Client	Confirmation dialog → `assignOffer`
`CompletionPanel`	Server + client button	Shows both flags ("ממתין לצד השני"), `confirmCompletion`
`ContactCard`	Server	Renders `get_counterpart_contact` result post-assignment
`RatingForm`, `Stars`	Client / Server	1–5 stars input; read-only display with average
`VerificationForm`	Client	Identity/professional application incl. doc upload
`AdminQueue`, `ModerationList`	Server + client actions	Approve/reject with note; hide/unhide; revoke
`GeolocationPrompt`	Client	One-time capture on `/profile`; writes via `updateProfile`
`MapView`, `RequestsMap` / `FeedMap`, `MapPicker`	Client	Leaflet maps: request location, feed pins + popups, click-to-choose location. Tiles are the one third-party runtime dependency — OpenStreetMap raster tiles: display-only, keyless, free; a tile-server outage degrades to a blank map square, never breaks a flow
`EmptyState`, `StatusChip`, `OfferPriceChip`, `Badge`	Server	Shared UI vocabulary; `OfferPriceChip` renders an offer's pricing stance (fixed price / volunteer / after-job)

Client components are leaves; every page is a Server Component that fetches data and passes plain props down. No context providers except the RTL/i18n-free strings module (plain imports).

---

8. State Management

- **URL = list state:** feed filters (category, distance) and page number are search params — shareable, back-button-correct, zero client cache.
- **Forms:** `useActionState(action)` per form; pending state from `useFormStatus`. After success, `revalidatePath` (concrete paths) refreshes every affected Server Component — no client store to reconcile.

- **Geolocation:** captured in a client component, persisted via `updateProfile`; the *server* is the source of truth for coordinates (the browser value is discarded after save).
- **Session:** `@supabase/ssr` cookies; middleware refreshes; components never hold auth state — they read it per request.

What does not exist: global stores, client-side data fetching (no SWR/React Query), optimistic updates (every mutation is followed by a server-rendered truth — at MVP latency this is simpler and always consistent).

---

9. Input Validation

One zod schema per form in `lib/validation/`, parsed authoritatively in the Server Action; client-side, native HTML constraint-validation attributes mirror the same bounds for instant feedback. DB constraints (§1.2) are the last line:

Schema	Rules (mirror of DB constraints)
`signUpSchema`	email format; password ≥ 8 chars
`profileSchema`	display_name 1–40; phone `^0\d{8,9}$` (optional until verification); lat/lng ranges, both-or-neither
`identityApplicationSchema`	full_name 2–60; self_description ≤ 500; phone required here (`^0\d{8,9}$` — mirrored by the `identity_requires_phone` CHECK); doc_path optional
`professionalApplicationSchema`	doc_path required (mirrored by the `professional_requires_doc` CHECK)
`requestSchema`	title 3–80; description 10–2000; category ∈ fixed list; lat/lng required (NOT NULL columns); photo paths 0–5 (optional)
`offerSchema`	message 5–1000; pricingMode ∈ {fixed, volunteer, after_job}; price required iff mode=fixed (0 < price ≤ 99999.99)
`finalPriceSchema`	0 < price ≤ 99999.99 — the after_job final amount set post-completion by the selected helper
`ratingSchema`	stars int 1–5; note ≤ 500
`reviewSchema` (admin)	note required on rejection, ≤ 500

File uploads validate client-side (type ∈ jpeg/png/webp, size ≤ 5 MB, count ≤ 5) and are re-bounded by bucket config server-side.

---

10. Error Handling

Four failure classes, each with one handling rule (details: architecture §10):

1. **Validation** → zod issues map to `fieldErrors`; rendered inline in Hebrew.
2. **Business/permission** → RPC codes and RLS denials map via the §5 table to a single friendly message; RLS-invisible rows render `not-found.tsx` (indistinguishable from truly missing — no existence leak).
3. **Infrastructure** (Supabase unreachable, storage failure) → `formError` "משהו השתבש, נסו שוב"; logged server-side with `console.error` (Vercel logs).
4. **Render errors** → route-group `error.tsx` boundaries with a reset button.

Upload flow failure semantics: photos upload first; if the subsequent action fails, orphaned objects may remain (invisible, bounded, cleanup in scale doc) — but a request row without photos can never exist (RPC).

---

11. UX Design (central experience)

- **The feed is the product's face:** card = photo thumbnail, title, category chip, distance chip, time-ago. One tap to detail. Filters as chips, not menus. RTL, `he` locale dates.
- **The request detail adapts to the viewer** (same URL, role-dependent panels): owner sees offers + assign buttons; a verified helper sees the offer form (or their existing offer + edit/withdraw); the assigned pair see the contact card
- completion panel with explicit "waiting for other side" state; post-completion the owner sees the rating form.
- **Verification is a funnel, not a wall:** unverified users browse freely; the gate appears exactly at the two action points (post / offer) as a redirect to `/verification` with a one-line explanation; pending state is always visible there ("הבקשה בבדיקה").
- **Trust signals are everywhere a helper appears:** badge (מאומת / בעל מקצוע) and star average render in every offer card and profile link.
- **Empty states teach:** empty feed → "פרסמו את הבקשה הראשונה"; no offers yet → what happens next; unrated completion → nudge to rate.
- **Loading:** route-level `loading.tsx` skeletons for feed and detail; form buttons show pending spinners via `useFormStatus`.
- **Accessibility basics:** semantic headings, labeled inputs, focus-visible, `dir="rtl"` at the root with logical-property spacing throughout.

---

12. Folder Structure (implementation-ready)

```
app/
  (public)/
    login/page.tsx  signup/page.tsx  emergency/page.tsx  page.tsx
  (app)/
    layout.tsx                      # session read, nav
    error.tsx  not-found.tsx        # route-group boundaries (§10)
    profile/page.tsx                # onboarding target — outside (onboarded)
                                    # to avoid a redirect loop
    (onboarded)/                    # onboarding gate group (redirects empty
      layout.tsx                    # display name → /profile)
      requests/page.tsx             # feed
      requests/loading.tsx          # skeletons (§11)
      requests/new/page.tsx
      requests/[id]/page.tsx
      requests/[id]/loading.tsx
      my/requests/page.tsx  my/offers/page.tsx
      helpers/[id]/page.tsx
      verification/page.tsx
      admin/page.tsx                # inside the shell: shares nav/session/error
                                    # boundary; RLS + in-page is_admin gate 404s
                                    # non-admins
  layout.tsx                        # <html dir="rtl" lang="he"> only
actions/
  auth.ts  profile.ts  verification.ts  requests.ts  offers.ts  ratings.ts
  admin.ts  helpers.ts
components/                         # §7 table
lib/
  supabase/{server,client,middleware}.ts
```

```
  geo.ts  strings.ts  categories.ts  errors.ts   # RPC-code → Hebrew mapping
  leaflet-icon.ts                  # Leaflet marker-icon asset wiring
  validation/{auth,profile,verification,request,offer,rating}.ts
supabase/
  migrations/                        # ordered SQL: enums, tables, indexes,
                                     # functions, triggers, RLS policies,
                                     # storage buckets, grants
scripts/
  seed.ts                          # demo data — see below
proxy.ts                           # session refresh (Next 16 name for root
                                   # middleware); matcher excludes /emergency
```

Seeding (demo/dev data): auth users cannot be reliably created by plain SQL (the `auth` schema is GoTrue-owned and its internals are unversioned), so `scripts/seed.ts` uses the **service-role admin API locally only** — exactly the "local tooling for migrations/seeding" carve-out of architecture §9 — to create users, then inserts domain rows. The demo dataset the presentation needs: 1 admin, 4 identity-verified users (1 with the professional badge), 1 unverified user (to demo the gate), requests in **every** lifecycle state (open, has_offers, assigned, completed, rated, cancelled, plus one hidden), offers in every status, and a few ratings so averages render.

---

13. Open Points Deliberately Deferred

Point	Where it lands
Orphaned storage objects cleanup	Scale doc (manual/cron sweep)
Rating aggregate denormalization	Scale doc (when profile pages get hot)
DB-side distance + keyset pagination	Scale doc (when the 200 cap shows)
SMS phone verification, ID-photo retention limits	Security doc roadmap
Email confirmation on signup	One-switch roadmap item (architecture §8.4)